LAB-2 Assignment
Date: 11/08/2026
Topic: Shell

Q1.1 (Predict the output) — Ordering without wait()

Given:

```
pid_t pid = fork();
if (pid == 0)
    printf("I am child. PID = %d\n", getpid());
else
    printf("I am parent. PID = %d\n", getpid());
```

Will "I am child" always print before "I am parent"? Why or why not?

Ans: No, "I am child" will not always print before "I am parent". After `fork()`, both the child and parent processes continue execution independently. Since there is no `wait()` or other synchronization mechanism, the scheduler decides which process runs first. Therefore, either the child or the parent may print first.

```
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/LAB 2$ vi q1_1.c
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/LAB 2$ gcc q1_1.c -o q1_1
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/LAB 2$ ./q1_1
I am parent. PID = 6380
I am child. PID = 6381
```

Q1.2 (Predict the output) — How many "Hello"s?

How many times does "Hello\n" get printed by this program?

```
int main() {
    fork();
    fork();
    printf("Hello\n");
    return 0;
}
```

Ans: `"Hello"` is printed **4 times**. Initially there is one process. After the first `fork()`, there are 2 processes. Both processes execute the second `fork()`, resulting in 4 processes. Each process executes the `printf()` statement once. Therefore, `"Hello"` is printed 4 times.
Output:
Hello
Hello
Hello

Hello

```
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/LAB 2$ gcc q1_2.c -o q1_2
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/LAB 2$ ./q1_2
Hello
Hello
Hello
Hello
```

Q2 (Code) — Write the child/parent branching pattern:
Write the skeleton of a program that forks, and in the child prints "I am child. PID = %d\n", in the parent prints "I am parent. PID = %d\n", and handles fork failure.

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main(void)
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }
    else if (pid == 0)
    {
        printf("I am child. PID = %d\n", getpid());
    }
    else
    {
        printf("I am parent. PID = %d\n", getpid());
    }

    return 0;
}
```

Output:

```
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/LAB 2$ gcc q2.c -o q2
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/LAB 2$ ./q2
I am parent. PID = 7284
I am child. PID = 7285
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/LAB 2$
```

Q3.1:(Code) - write the code for execute the “cat” command.

Hint: check the warmup codes we talk in the lab parta.c file

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
   if (argc != 2)
   {
      printf("Usage: %s <filename>\n", argv[0]);
      return 1;
   }

   printf("Executing cat command:\n");

   execlp("cat", "cat", argv[1], (char *)NULL);

   // If exec fails, this line will execute
   perror("exec failed");

   return 1;
}
```

```
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/LAB 2$ nano q3_1.c
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/LAB 2$ gcc q3_1.c -o q3_1
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/LAB 2$ echo "Hello, this is a test file." > test.txt
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/LAB 2$ ./q3_1 test.txt
Executing cat command:
Hello, this is a test file.
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/LAB 2$
```

Q3.2: (Code) - runcmd.c (fork + exec + wait combined)

Write a program that takes exactly two command-line arguments — a command and one argument to it (e.g. sleep 10) — forks a child, execs the command in the child, and has the parent wait, reap, and report success. It must print an error for the wrong number of arguments.

Hints to write the program: Your program needs to do **four main things**:

1. **Check command-line arguments**
   - argc tells you how many arguments were supplied
2. Create a child process
   - fork() has three possible results:
     - < 0 → process creation failed
     - == 0 → you are inside the **child**
     - > 0 → you are inside the **parent**
3. In the child, execute the requested command
4. In the parent, wait for the child

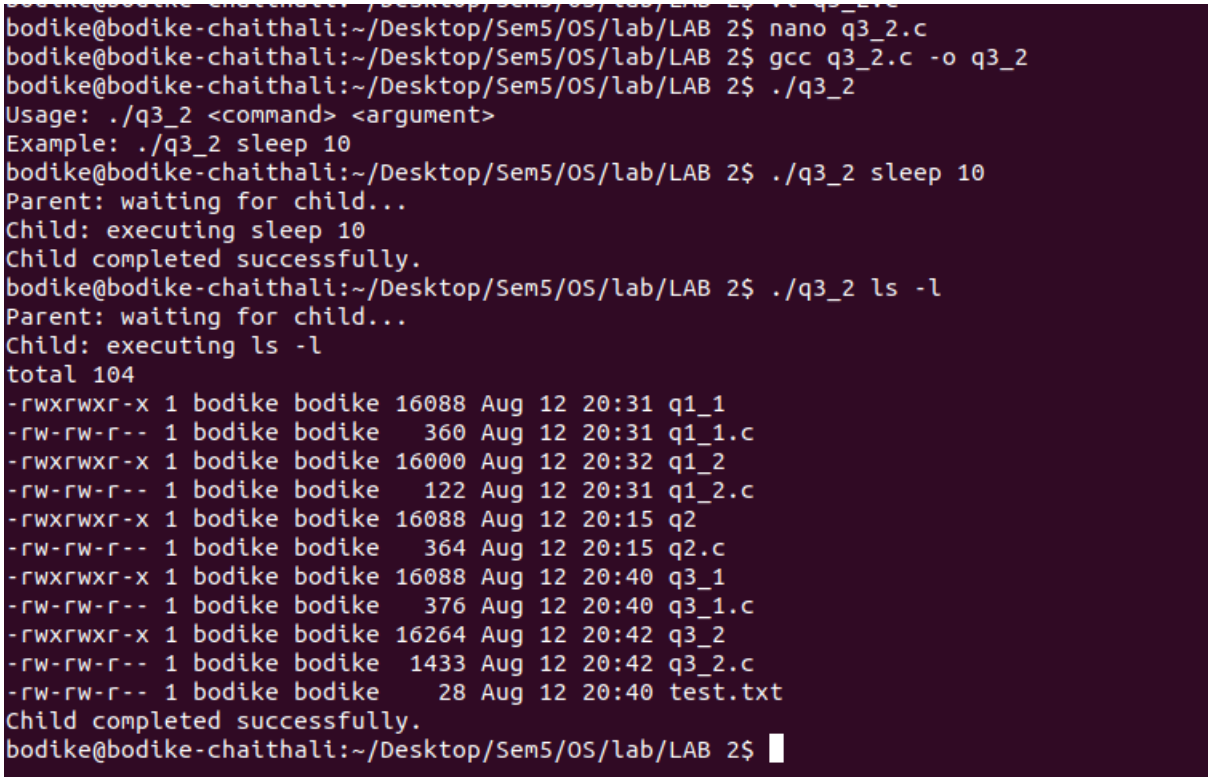
Code:

```
#include <stdio.h>
```

```c
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    pid_t pid;
    int status;

    /* Check command-line arguments */
    if (argc != 3)
    {
        printf("Usage: %s <command> <argument>\n", argv[0]);
        printf("Example: %s sleep 10\n", argv[0]);
        return 1;
    }

    /* Create child process */
    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    /* Child process */
    if (pid == 0)
    {
        printf("Child: executing %s %s\n", argv[1], argv[2]);

        execvp(argv[1], &argv[1]);

        /* This executes only if execvp fails */
        perror("exec failed");
        exit(1);
    }

    /* Parent process */
    else
    {
        printf("Parent: waiting for child...\n");

        if (waitpid(pid, &status, 0) == -1)
        {
            perror("waitpid failed");
            return 1;
```

```
    }

    /* Check whether child exited normally */
    if (WIFEXITED(status))
    {
        if (WEXITSTATUS(status) == 0)
        {
            printf("Child completed successfully.\n");
        }
        else
        {
            printf("Child exited with status %d.\n", WEXITSTATUS(status));
        }
    }
    else
    {
        printf("Child did not exit normally.\n");
    }
  }

  return 0;
}
```

Output:



Q4 (Code):Background execution:

Now, we will extend the shell to support background execution of processes. Extend your

shell program of part A in the following manner: if a Linux command is followed by &, the command must be executed in the background. That is, the shell must start the execution of the command, and return to prompt the user for the next input, without waiting for the previous command to complete. The output of the command can get printed to the shell as and when it appears. A command not followed by & must simply execute in the foreground as before.

You can assume that the commands running in the background are simple Linux commands without pipes or redirections or any other special case handling like cd. You can assume that the user will enter only one foreground or background command at a time on the command prompt, and the command and & are separated by a space. You may assume that there are no more than 64 background commands executing at any given time. A helpful tip for testing: use long running commands like sleep to test your foreground and background implementations, as such commands will give you enough time to run ps in another window to check that the commands are executing as specified. Across both background and foreground execution, ensure that the shell reaps all its children that have terminated. Unlike in the case of foreground execution, the background processes can be reaped with a time delay. For example, the shell may check for dead children periodically, say, when it obtains a new user input from the terminal. When the shell reaps a terminated background process, it must print a message Shell: Background process finished to let the user know that a background process has finished. Hint: you may use a variant of the wait system call, e.g., waitpid(-1, NULL, WNOHANG) to reap background processes without blocking. Read up more on how to invoke wait without blocking.

You must test your implementation for the cases where background and foreground processes are running together, and ensure that dead children are being reaped correctly in such cases. Recall that a generic wait system call can reap and return any dead child. So if you are waiting for a foreground process to terminate and invoke wait, it may reap and return a terminated background process. In that case, you must not erroneously return to the command prompt for the next command, but you must wait for the foreground command to terminate as well. To avoid such confusions, you may choose to use the waitpid variant of this system call, to be sure that you are reaping the correct foreground child. Once again, use long running commands like sleep, run ps in another window, and monitor the execution of your processes, to thoroughly test your shell with a combination of background and foreground processes. In particular, test that a background process finishing up in the middle of a foreground command execution will not cause your shell to incorrectly return to the command prompt before the foreground command finishes.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT_SIZE 1024
#define MAX_TOKEN_SIZE 64
#define MAX_NUM_TOKENS 64
```

```
/* Tokenize input */
char **tokenize(char *line)
{
    char **tokens = (char **)malloc(MAX_NUM_TOKENS * sizeof(char *));
    char *token = (char *)malloc(MAX_TOKEN_SIZE * sizeof(char));

    int i, tokenIndex = 0, tokenNo = 0;

    for (i = 0; i < strlen(line); i++)
    {
        char c = line[i];

        if (c == ' ' || c == '\t' || c == '\n')
        {
            token[tokenIndex] = '\0';

            if (tokenIndex != 0)
            {
                tokens[tokenNo] = (char *)malloc(MAX_TOKEN_SIZE);
                strcpy(tokens[tokenNo], token);
                tokenNo++;
                tokenIndex = 0;
            }
        }
        else
        {
            token[tokenIndex++] = c;
        }
    }

    free(token);
    tokens[tokenNo] = NULL;

    return tokens;
}

/* Reap terminated background processes */
void reap_background_processes()
{
    int status;
    pid_t pid;

    while ((pid = waitpid(-1, &status, WNOHANG)) > 0)
    {
        printf("Shell: Background process finished\n");
    }
}
```

```
int main()
{
    char line[MAX_INPUT_SIZE];
    char cwd[256];
    char **tokens;

    while (1)
    {
        /*
         * Reap any background processes that have already
         * finished. WNOHANG ensures that the shell does not wait.
         */
        reap_background_processes();

        /* Display current working directory */
        getcwd(cwd, sizeof(cwd));
        printf("%s $ ", cwd);
        fflush(stdout);

        if (fgets(line, sizeof(line), stdin) == NULL)
            break;

        /* Ignore empty command */
        if (strcmp(line, "\n") == 0)
            continue;

        tokens = tokenize(line);

        if (tokens[0] == NULL)
        {
            free(tokens);
            continue;
        }

        /*
         * Check whether command is a background command.
         * The last token must be "&".
         */
        int background = 0;
        int tokenCount = 0;

        while (tokens[tokenCount] != NULL)
        {
            tokenCount++;
        }

        if (tokenCount > 1 &&
            strcmp(tokens[tokenCount - 1], "&") == 0)
```

```
{
    background = 1;

    /* Remove "&" from arguments passed to execvp */
    free(tokens[tokenCount - 1]);
    tokens[tokenCount - 1] = NULL;
}

/* Built-in: cd */
if (strcmp(tokens[0], "cd") == 0)
{
    if (background)
    {
        printf("cd cannot be executed in background\n");
    }
    else if (tokens[1] == NULL || tokens[2] != NULL)
    {
        printf("Usage: cd <directory>\n");
    }
    else
    {
        if (chdir(tokens[1]) != 0)
            perror("cd");
    }

    for (int i = 0; tokens[i] != NULL; i++)
        free(tokens[i]);

    free(tokens);

    continue;
}

/* Create child process */
pid_t pid = fork();

if (pid < 0)
{
    perror("fork");

    for (int i = 0; tokens[i] != NULL; i++)
        free(tokens[i]);

    free(tokens);

    continue;
}
```

```
/* Child process */
if (pid == 0)
{
    execvp(tokens[0], tokens);

    /* exec failed */
    perror(tokens[0]);
    exit(1);
}

/* Parent process */

if (background)
{
    /*
     * Background process:
     * Do NOT wait for the child.
     * Immediately return to the prompt.
     */
    printf("Shell: Background process started (PID = %d)\n", pid);
}
else
{
    /*
     * Foreground process:
     * Wait specifically for this child.
     *
     * This is important because waitpid(pid, ...)
     * cannot accidentally reap a background child.
     */
    int status;

    if (waitpid(pid, &status, 0) == -1)
    {
        perror("waitpid");
    }
    else if (WIFEXITED(status))
    {
        printf("EXITSTATUS: %d\n", WEXITSTATUS(status));
    }
}

/* Free memory */
for (int i = 0; tokens[i] != NULL; i++)
    free(tokens[i]);

free(tokens);
```

```
        /*
         * Reap background processes that may have finished
         * while the foreground command was running.
         */
        reap_background_processes();
    }

    /*
     * Final cleanup: reap any children that have already
     * terminated without blocking.
     */
    reap_background_processes();

    return 0;
}
```

output:

```
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/LAB 2$ nano partb.c
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/LAB 2$ gcc partb.c -o partb
bodike@bodike-chaithali:~/Desktop/Sem5/OS/lab/LAB 2$ ./partb
/home/bodike/Desktop/Sem5/OS/lab/LAB 2 $ sleep 2
EXITSTATUS: 0
/home/bodike/Desktop/Sem5/OS/lab/LAB 2 $ sleep 10 &
Shell: Background process started (PID = 14423)
/home/bodike/Desktop/Sem5/OS/lab/LAB 2 $ ps
    PID TTY          TIME CMD
   6288 pts/0    00:00:00 bash
  14312 pts/0    00:00:00 partb
  14423 pts/0    00:00:00 sleep <defunct>
  14460 pts/0    00:00:00 ps
EXITSTATUS: 0
Shell: Background process finished
/home/bodike/Desktop/Sem5/OS/lab/LAB 2 $ sleep 10 &
Shell: Background process started (PID = 14498)
/home/bodike/Desktop/Sem5/OS/lab/LAB 2 $ ps
    PID TTY          TIME CMD
   6288 pts/0    00:00:00 bash
  14312 pts/0    00:00:00 partb
  14498 pts/0    00:00:00 sleep
  14534 pts/0    00:00:00 ps
EXITSTATUS: 0
/home/bodike/Desktop/Sem5/OS/lab/LAB 2 $ sleep 3 &
Shell: Background process started (PID = 14571)
Shell: Background process finished
/home/bodike/Desktop/Sem5/OS/lab/LAB 2 $ ls
partb  partb.c  q1_1  q1_1.c  q1_2  q1_2.c  q2  q2.c  q3_1  q3_1.c  q3_2  q
EXITSTATUS: 0
Shell: Background process finished
/home/bodike/Desktop/Sem5/OS/lab/LAB 2 $ sleep 5 &
Shell: Background process started (PID = 14646)
/home/bodike/Desktop/Sem5/OS/lab/LAB 2 $ sleep 10
EXITSTATUS: 0
Shell: Background process finished
```